Multi-Head Self-Attention from first principles

Part 1 · Systems Architect — Rafid Mohammad Ahsan · file `01_MultiHead_Attention_Core.py` · run: `python 01_MultiHead_Attention_Core.py`

Pure PyTorch `MultiHeadSelfAttention`: learned W_Q, W_K, W_V, W_O; head split (B, N, h, d) → (B, h, N, d); hand-written numerically stable softmax; causal and padding masks; empirical variance-preservation test and a study of softmax saturation without 1/√d_k. No `nn.MultiheadAttention` or fused attention kernels are used.

Run output

```
$ python 01_MultiHead_Attention_Core.py
================================================================================
USSE NOOSPHERE · SUBSYSTEM 03 · PART 1: MULTI-HEAD SELF-ATTENTION

=== Subsystem 03 · Variance preservation test (q, k ~ N(0,1)) ===
  d_k |   Var(q·k) | expected |  Var(q·k/√d_k) | result
------------------------------------------------------------
   16 |      16.31 |       16 |         1.0195 | PASS
   32 |      32.14 |       32 |         1.0044 | PASS
   64 |      64.18 |       64 |         1.0027 | PASS
  128 |     128.13 |      128 |         1.0010 | PASS
  256 |     255.96 |      256 |         0.9998 | PASS
  512 |     511.99 |      512 |         1.0000 | PASS
[PASS] Scaling by 1/sqrt(d_k) keeps Var ≈ 1 for every head size.

=== What happens WITHOUT scaling: softmax saturation ===
  d_k | max weight (scaled) | max weight (unscaled) | ‖J‖ scaled | ‖J‖ unscaled
-------------------------------------------------------------------------------
   16 |               0.069 |                 0.533 |     0.1351 |       0.2943
   64 |               0.069 |                 0.769 |     0.1373 |       0.2010
  256 |               0.069 |                 0.888 |     0.1359 |       0.1335
  512 |               0.068 |                 0.921 |     0.1363 |       0.0919
Scaled: the softmax behaves the same for every d_k. Unscaled: scores grow like sqrt(d_k),
the softmax drifts towards one-hot and its Jacobian (the gradient path) keeps shrinking.

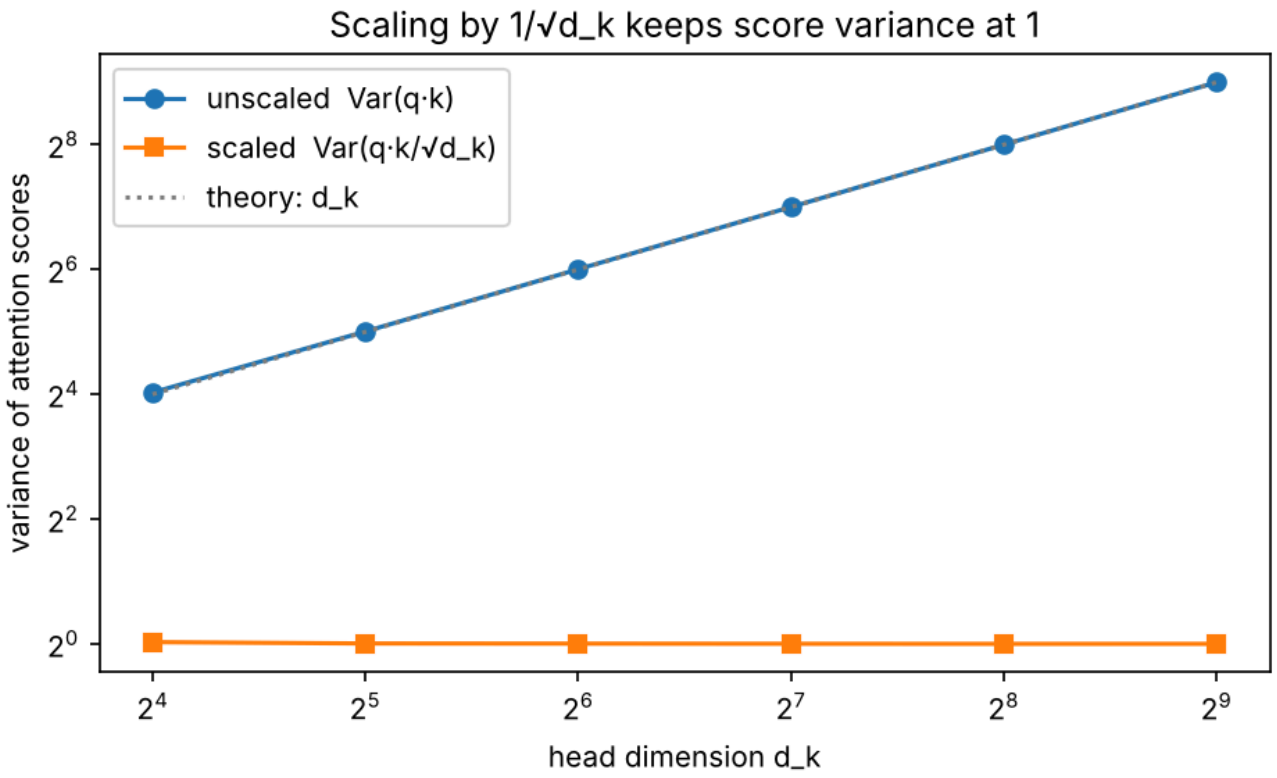
=== Causal mask check ===
max attention weight above diagonal: 0.0e+00  ·  outputs 0-6 unchanged after editing tokens 7-9: True
[PASS] No information flows from future tokens.

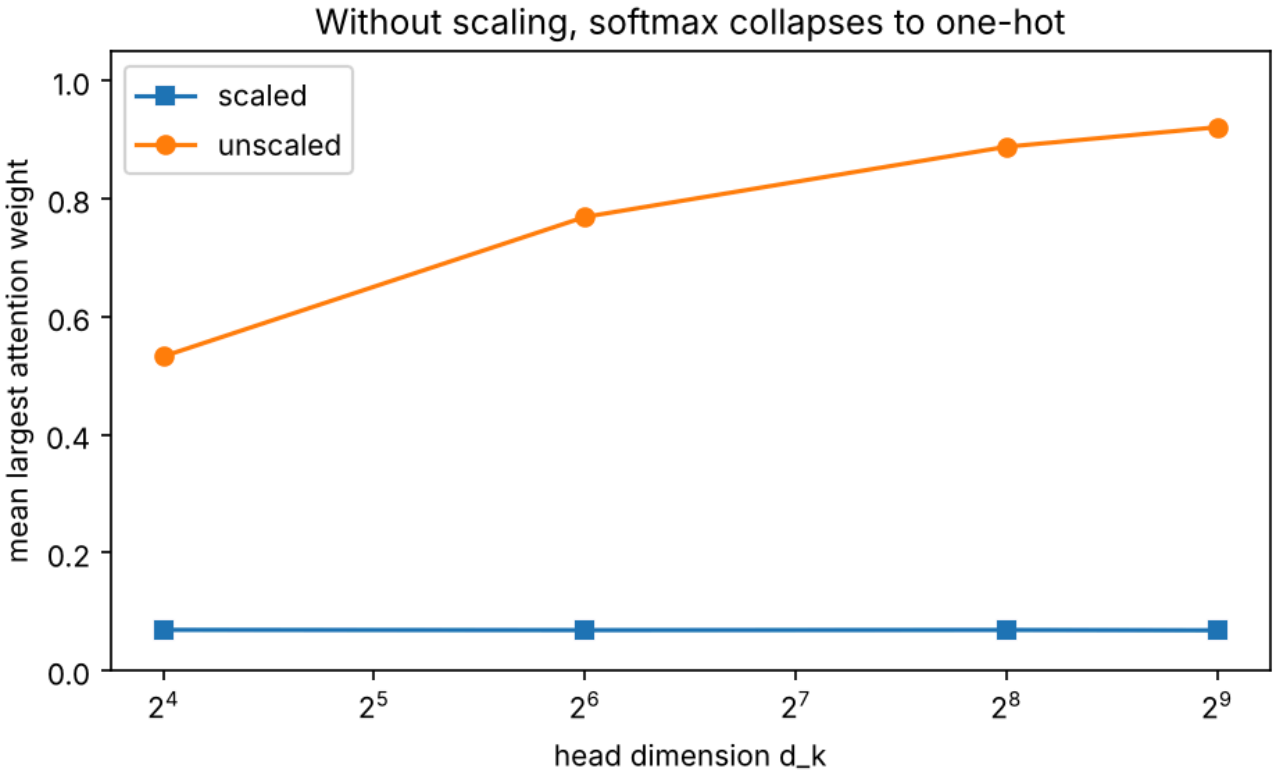
=== Forward pass ===
input (2, 16, 256) -> output (2, 16, 256) · attention (2, 4, 16, 16) · rows sum to 1: True
parameters: 4 × d_model² = 262,144

Figures written to figures/01_variance_scaling.png, figures/01_softmax_saturation.png

[OK] Subsystem 03 attention mesh online.
```

Figures





Source code

```
"""
Mission 03 — Part 1: Systems Architect (Lead Engineer)
Multi-Head Self-Attention from first principles, causal masking, variance preservation.

USSE Noosphere · Sector SEC-03-M45 (Pleiades) · Subsystem 03: Multi-Head Attention Mesh
Squadron Colby
```

```
Run:  python 01_MultiHead_Attention_Core.py
      (writes figures/01_variance_scaling.png and figures/01_softmax_saturation.png)

Only basic tensor ops are used: no nn.MultiheadAttention, no F.scaled_dot_product_attention.
"""
from __future__ import annotations

import math
import pathlib
from typing import Optional, Tuple

import torch
import torch.nn as nn

FIG_DIR = pathlib.Path(__file__).resolve().parent / "figures"


# ---------------------------------------------------------------------------- #
# 1. Building blocks
# ---------------------------------------------------------------------------- #

def causal_mask(seq_len: int, device=None) -> torch.Tensor:
    """Additive causal mask: 0 on and below the diagonal, -inf above it.

    Position i may only attend to positions j <= i (no peeking at future tokens).
    Shape (seq_len, seq_len); broadcasts over (batch, heads).
    """
    return torch.triu(torch.full((seq_len, seq_len), float("-inf"), device=device), diagonal=1)


def softmax(x: torch.Tensor, dim: int = -1) -> torch.Tensor:
    """Numerically stable softmax written out by hand (subtract the row max first).

    Rows that are entirely -inf (fully masked) return zeros instead of NaN.
    """
    x_max = x.amax(dim=dim, keepdim=True)
    x_max = torch.where(torch.isinf(x_max), torch.zeros_like(x_max), x_max)
    e = torch.exp(x - x_max)
    return e / e.sum(dim=dim, keepdim=True).clamp_min(1e-30)


def scaled_dot_product_attention(q: torch.Tensor, k: torch.Tensor, v: torch.Tensor,
                                 mask: Optional[torch.Tensor] = None,
                                 scale: bool = True) -> Tuple[torch.Tensor, torch.Tensor]:
    """Attention(Q, K, V) = softmax(Q K^T / sqrt(d_k) + mask) V.

    q, k, v: (..., seq, d_k). Returns (context, weights).
    `scale=False` is only used to demonstrate what goes wrong without 1/sqrt(d_k).
    """
    d_k = q.size(-1)
    scores = q @ k.transpose(-2, -1)
    if scale:
        scores = scores / math.sqrt(d_k)
    if mask is not None:
        scores = scores + mask
    weights = softmax(scores, dim=-1)
    return weights @ v, weights


# ---------------------------------------------------------------------------- #
# 2. Multi-Head Self-Attention module
# ---------------------------------------------------------------------------- #

class MultiHeadSelfAttention(nn.Module):
    """Multi-head self-attention with learned W_Q, W_K, W_V, W_O in R^{d_model x d_model}.

    Shapes: x (B, N, d_model) -> project -> (B, N, h, d_head) -> transpose -> (B, h, N, d_head)
            -> attention per head -> concat heads -> W_O -> (B, N, d_model)
    """

    def __init__(self, d_model: int = 512, num_heads: int = 8, dropout: float = 0.0,
                 bias: bool = False) -> None:
        super().__init__()
        if d_model % num_heads != 0:
            raise ValueError(f"d_model ({d_model}) must be divisible by num_heads ({num_heads})")
        self.d_model, self.num_heads = d_model, num_heads
        self.head_dim = d_model // num_heads
        self.w_q = nn.Linear(d_model, d_model, bias=bias)
        self.w_k = nn.Linear(d_model, d_model, bias=bias)
        self.w_v = nn.Linear(d_model, d_model, bias=bias)
```

```
        self.w_o = nn.Linear(d_model, d_model, bias=bias)
        self.dropout = nn.Dropout(dropout)

    def split_heads(self, t: torch.Tensor) -> torch.Tensor:
        """(B, N, d_model) -> (B, N, h, d_head) -> (B, h, N, d_head)."""
        b, n, _ = t.shape
        return t.view(b, n, self.num_heads, self.head_dim).transpose(1, 2)

    def merge_heads(self, t: torch.Tensor) -> torch.Tensor:
        """(B, h, N, d_head) -> (B, N, h, d_head) -> (B, N, d_model)."""
        b, h, n, d = t.shape
        return t.transpose(1, 2).contiguous().view(b, n, h * d)

    def forward(self, x: torch.Tensor, causal: bool = True,
                padding_mask: Optional[torch.Tensor] = None) -> Tuple[torch.Tensor, torch.Tensor]:
        """
        x:            (B, N, d_model)
        causal:       apply the lower-triangular mask (decoder / autoregressive mode)
        padding_mask: optional bool (B, N), True = real token, False = padding (masked as a key)
        Returns (output (B, N, d_model), attention weights (B, h, N, N)).
        """
        _, n, _ = x.shape
        q = self.split_heads(self.w_q(x))
        k = self.split_heads(self.w_k(x))
        v = self.split_heads(self.w_v(x))

        mask = causal_mask(n, x.device) if causal else None
        if padding_mask is not None:
            pad = torch.zeros(padding_mask.shape, device=x.device).masked_fill(~padding_mask,
                                                                               float("-inf"))
            pad = pad[:, None, None, :]                       # (B, 1, 1, N): mask keys
            mask = pad if mask is None else mask + pad

        context, weights = scaled_dot_product_attention(q, k, v, mask)
        context = self.dropout(context)
        return self.w_o(self.merge_heads(context)), weights


# ---------------------------------------------------------------------------- #
# 3. Verification: variance preservation and softmax saturation
# ---------------------------------------------------------------------------- #

def dot_product_variance(d_k: int, n: int = 128, batch: int = 32, seed: int = 42) -> Tuple[float, float]:
    """Empirical Var(q·k) unscaled vs scaled, with q, k ~ N(0, 1) i.i.d.

    Theory: q·k = sum_i q_i k_i, each term has mean 0 and variance 1, so Var = d_k.
    Dividing by sqrt(d_k) divides the variance by d_k -> 1.
    """
    g = torch.Generator().manual_seed(seed)
    q = torch.randn(batch, n, d_k, generator=g)
    k = torch.randn(batch, n, d_k, generator=g)
    dots = q @ k.transpose(-2, -1)
    return dots.var().item(), (dots / math.sqrt(d_k)).var().item()


def softmax_stats(d_k: int, n: int = 128, scale: bool = True, seed: int = 0) -> Tuple[float, float]:
    """(mean max attention weight, mean softmax-Jacobian norm) for random q, k.

    A max weight near 1.0 means the softmax is one-hot (saturated); the Jacobian
    d softmax / d scores = diag(p) - p p^T then goes to 0, so gradients vanish.
    """
    g = torch.Generator().manual_seed(seed)
    q = torch.randn(64, n, d_k, generator=g)
    k = torch.randn(64, n, d_k, generator=g)
    scores = q @ k.transpose(-2, -1)
    if scale:
        scores = scores / math.sqrt(d_k)
    p = softmax(scores)                                       # (64, n, n)
    max_w = p.amax(-1).mean().item()
    # Frobenius norm of J = diag(p) - p p^T for each row, averaged
    p_rows = p.reshape(-1, n)[:256]
    jac = torch.diag_embed(p_rows) - p_rows[:, :, None] * p_rows[:, None, :]
    return max_w, jac.flatten(1).norm(dim=1).mean().item()


def verify_variance_preservation(dims=(16, 32, 64, 128, 256, 512), tol: float = 0.1) -> list[dict]:
    print("=== Subsystem 03 · Variance preservation test (q, k ~ N(0,1)) ===")
    print(f"{'d_k':>5} | {'Var(q·k)':>10} | {'expected':>8} | {'Var(q·k/√d_k)':>14} | result")
    print("-" * 58)
    rows = []
```

```
    for d in dims:
        raw, scaled = dot_product_variance(d)
        ok = abs(scaled - 1.0) < tol
        rows.append({"d_k": d, "var_unscaled": raw, "var_scaled": scaled, "pass": ok})
        print(f"{d:>5} | {raw:>10.2f} | {d:>8} | {scaled:>14.4f} | {'PASS' if ok else 'FAIL'}")
    assert all(r["pass"] for r in rows), "variance preservation failed"
    print("[PASS] Scaling by 1/sqrt(d_k) keeps Var ≈ 1 for every head size.\n")
    return rows


def verify_softmax_saturation(dims=(16, 64, 256, 512)) -> list[dict]:
    print("=== What happens WITHOUT scaling: softmax saturation ===")
    print(f"{'d_k':>5} | {'max weight (scaled)':>19} | {'max weight (unscaled)':>21} | "
          f"{'‖J‖ scaled':>10} | {'‖J‖ unscaled':>12}")
    print("-" * 82)
    rows = []
    for d in dims:
        ms, js = softmax_stats(d, scale=True)
        mu, ju = softmax_stats(d, scale=False)
        rows.append({"d_k": d, "max_w_scaled": ms, "max_w_unscaled": mu,
                     "jac_scaled": js, "jac_unscaled": ju})
        print(f"{d:>5} | {ms:>19.3f} | {mu:>21.3f} | {js:>10.4f} | {ju:>12.4f}")
    print("Scaled: the softmax behaves the same for every d_k. Unscaled: scores grow like sqrt(d_k),\n"
          "the softmax drifts towards one-hot and its Jacobian (the gradient path) keeps shrinking.\n")
    return rows


def verify_causal_mask(seed: int = 7) -> None:
    """Changing a future token must not change earlier outputs; weights above the diagonal = 0."""
    torch.manual_seed(seed)
    mha = MultiHeadSelfAttention(d_model=64, num_heads=4).eval()
    x = torch.randn(1, 10, 64)
    y1, w = mha(x, causal=True)
    x2 = x.clone()
    x2[:, 7:] = torch.randn(1, 3, 64)                       # change tokens 7..9
    y2, _ = mha(x2, causal=True)
    upper = torch.triu(w[0, 0], diagonal=1).abs().max().item()
    same_past = torch.allclose(y1[:, :7], y2[:, :7], atol=1e-6)
    print("=== Causal mask check ===")
    print(f"max attention weight above diagonal: {upper:.1e}  ·  outputs 0-6 unchanged after "
          f"editing tokens 7-9: {same_past}")
    assert upper == 0.0 and same_past
    print("[PASS] No information flows from future tokens.\n")


def plot_figures(var_rows: list[dict], sat_rows: list[dict]) -> None:
    try:
        import matplotlib
        matplotlib.use("Agg")
        import matplotlib.pyplot as plt
    except ImportError:
        print("(matplotlib not installed: skipping figures)")
        return
    FIG_DIR.mkdir(exist_ok=True)
    d = [r["d_k"] for r in var_rows]
    fig, ax = plt.subplots(figsize=(6.4, 4))
    ax.plot(d, [r["var_unscaled"] for r in var_rows], "o-", label="unscaled  Var(q·k)")
    ax.plot(d, [r["var_scaled"] for r in var_rows], "s-", label="scaled  Var(q·k/√d_k)")
    ax.plot(d, d, ":", color="grey", label="theory: d_k")
    ax.set_xscale("log", base=2); ax.set_yscale("log", base=2)
    ax.set_xlabel("head dimension d_k"); ax.set_ylabel("variance of attention scores")
    ax.set_title("Scaling by 1/√d_k keeps score variance at 1")
    ax.legend(); fig.tight_layout(); fig.savefig(FIG_DIR / "01_variance_scaling.png", dpi=150)
    plt.close(fig)

    d = [r["d_k"] for r in sat_rows]
    fig, ax = plt.subplots(figsize=(6.4, 4))
    ax.plot(d, [r["max_w_scaled"] for r in sat_rows], "s-", label="scaled")
    ax.plot(d, [r["max_w_unscaled"] for r in sat_rows], "o-", label="unscaled")
    ax.set_xscale("log", base=2); ax.set_ylim(0, 1.05)
    ax.set_xlabel("head dimension d_k"); ax.set_ylabel("mean largest attention weight")
    ax.set_title("Without scaling, softmax collapses to one-hot")
    ax.legend(); fig.tight_layout(); fig.savefig(FIG_DIR / "01_softmax_saturation.png", dpi=150)
    plt.close(fig)
    print("Figures written to figures/01_variance_scaling.png, figures/01_softmax_saturation.png\n")


def main() -> None:
    print("USSE NOOSPHERE · SUBSYSTEM 03 · PART 1: MULTI-HEAD SELF-ATTENTION\n")
    var_rows = verify_variance_preservation()
```

```
    sat_rows = verify_softmax_saturation()
    verify_causal_mask()

    torch.manual_seed(0)
    mha = MultiHeadSelfAttention(d_model=256, num_heads=4).eval()
    x = torch.randn(2, 16, 256)
    out, attn = mha(x, causal=True)
    rows_sum_to_one = torch.allclose(attn.sum(-1), torch.ones_like(attn.sum(-1)), atol=1e-5)
    print("=== Forward pass ===")
    print(f"input {tuple(x.shape)} -> output {tuple(out.shape)} · attention {tuple(attn.shape)} · "
          f"rows sum to 1: {rows_sum_to_one}")
    params = sum(p.numel() for p in mha.parameters())
    print(f"parameters: 4 × d_model² = {params:,}\n")
    plot_figures(var_rows, sat_rows)
    print("[OK] Subsystem 03 attention mesh online.")


if __name__ == "__main__":
    main()
```